

**Московский авиационный институт  
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной математики**

**Кафедра вычислительной математики и программирования**

**Лабораторная работа №3-4 по курсу «Информационный поиск»**

Студент: М.А. Пирязев  
Преподаватель: А.А. Кухтичев  
Группа: М8О-207Б-22  
Дата: 20.12.2025  
Оценка:  
Подпись:

**Москва, 2025**

## Оглавление

|                                                                   |    |
|-------------------------------------------------------------------|----|
| <b>1. Лабораторная работа №6 “Сжатие”</b> .....                   | 2  |
| <b>1.1 Описание</b> .....                                         | 2  |
| <b>2. Исходный код</b> .....                                      | 2  |
| <b>2.1 Выбранный метод сжатия</b> .....                           | 2  |
| <b>2.2 Побитовая схема хранения</b> .....                         | 2  |
| <b>2.3 Схема хранения постингов и координат</b> .....             | 4  |
| <b>2.4 Причины выбора метода сжатия</b> .....                     | 4  |
| <b>2.5 Частота термов</b> .....                                   | 4  |
| <b>2.6 Корректность поиска после внедрения сжатия</b> .....       | 5  |
| <b>2.7 Тестирование</b> .....                                     | 6  |
| <b>2.8 Проверка построения индексов и сравнение времени</b> ..... | 11 |
| <b>2.9 Сравнение результатов поиска</b> .....                     | 13 |
| <b>3. Выводы</b> .....                                            | 16 |
| <b>Список литературы</b> .....                                    | 17 |

# **1. Лабораторная работа №6 “Сжатие”**

## **1.1 Описание**

Целью лабораторной работы является внедрение сжатия в инвертированный индекс и анализ влияния сжатия на объём хранимых данных и на скорость прохождения по координатным спискам при выполнении поиска. В рамках работы реализованы построение сжатого инвертированного индекса и утилита булевого поиска, работающая поверх данного формата индекса.

## **2. Исходный код**

### **2.1 Выбранный метод сжатия**

В качестве метода сжатия выбран подход, основанный на двух идеях: кодирование разностей (gap encoding) и переменного-байтового кодирования целых чисел (variable byte coding, vbyte). Списки документов для каждого термина хранятся в виде возрастающих идентификаторов документов, но на диск записываются не сами docid, а разности между соседними docid, то есть doc\_gap. Аналогичный подход применяется к координатам вхождений в документе: позиции pos записываются как pos\_gap.

### **2.2 Побитовая схема хранения**

Переменного-байтовое кодирование (vbyte) применяется для компактного хранения неотрицательных целых чисел в виде последовательности байтов. Каждый байт в таком представлении делится на две части: младшие 7 бит содержат фрагмент числа, а старший бит служит признаком продолжения. Если старший бит равен 1, это означает, что число не закончилось и следующий байт тоже относится к нему. Если старший бит равен 0, то текущий байт является последним и декодирование данного числа можно завершить.

На практике это удобно тем, что маленькие числа занимают всего один байт, а большие автоматически расширяются до двух и более байтов. При декодировании байты

читаются последовательно, из каждого извлекаются 7 полезных бит и добавляются к результату со сдвигом на 0, 7, 14 и так далее, пока не встретится байт, у которого старший бит равен 0. За счёт такой схемы не требуется хранить длину числа отдельно: конец определяется содержимым потока.

Пример: число 300 можно разбить на 7-битные группы (начиная с младших бит) как 44 и 2. Тогда первый байт будет содержать 44 и признак продолжения (в hex это AC), а второй байт будет содержать 2 без признака продолжения (в hex это 02). В итоге число 300 кодируется последовательностью байтов AC 02.

Ниже приведён фрагмент из построения сжатого индекса, где значения последовательно записываются в байтовый массив постингов через `lab6::vbyte_put_u32`. Сначала записывается разность между текущим и предыдущим идентификатором документа в списке, затем частота термина в документе, затем разности позиций вхождений.

```
const std::uint32_t doc_gap = doc - prev_doc;
prev_doc = doc;
lab6::vbyte_put_u32(postings, doc_gap);
const std::uint32_t tf = static_cast<std::uint32_t>(j - i);
lab6::vbyte_put_u32(postings, tf);
std::uint32_t prev_pos = 0;
for (std::size_t k = i; k < j; ++k) {
    const std::uint32_t pos = occ.data[k].pos;
    const std::uint32_t pos_gap = pos - prev_pos;
    prev_pos = pos;
    lab6::vbyte_put_u32(postings, pos_gap);
}
```

## 2.3 Схема хранения постингов и координат

Данные по каждому терму записываются последовательным потоком целых чисел, закодированных vbyte. Логическая структура для одного терма: - далее многократно повторяется блок по документу: - разность идентификатора документа относительно предыдущего документа в списке (gap по документам); - частота терма в документе (сколько раз терм встретился в этом документе); - затем список позиций вхождений, но в виде разностей относительно предыдущей позиции (gap по позициям). Такое представление удобно тем, что поток можно декодировать последовательно, не требуя случайного доступа к середине блока для корректного восстановления значений.

## 2.4 Причины выбора метода сжатия

Во-первых, инвертированный индекс по определению содержит отсортированные списки идентификаторов документов, а значит разности между соседними значениями часто существенно меньше самих значений. Для позиций внутри документа ситуация аналогичная: позиции возрастают, а разности обычно малы.

Во-вторых, vbyte является простым и устойчивым методом: он не требует сложных таблиц, не привязан к размеру блока и позволяет быстро декодировать значения в потоковом режиме. Это важно, потому что во время поиска требуется многократно проходить по спискам, объединять их булевыми операциями и получать пересечения или объединения.

В-третьих, данный метод хорошо подходит для практической реализации в рамках лабораторной работы: он даёт измеримый выигрыш по размеру и остаётся достаточно прозрачным для отладки и проверки корректности.

## 2.5 Частота термов

Эффект сжатия зависит от частотности терма, так как меняются длина списков и распределение разностей. Для **редких** термов число документов в списке мало. Размер уменьшается, потому что даже небольшое количество значений хранится компактно, а

большинство разностей по документам кодируются одним байтом. Скорость прохождения, как правило, почти не ухудшается, поскольку декодировать нужно мало чисел, и накладные расходы несущественны. Для термов **средней** частотности выигрыш по размеру наиболее заметен в практическом смысле, потому что такие термы встречаются часто в запросах и формируют значительную часть общего объёма постингов. Сжатие снижает объём чтения с диска и уменьшает количество данных, проходящих через кэш, поэтому проход по спискам часто ускоряется даже с учётом необходимости декодирования.

Доля затрат на декодирование растёт, но обычно компенсируется меньшим объёмом ввода-вывода. Для **высокочастотных** термов списки максимально длинные, а число координатных записей (позиции вхождений) может быть очень большим. По размеру сжатие даёт сильный эффект, потому что абсолютные значения и позиции заменяются более короткими разностями, и большое число записей кодируется относительно компактно. По скорости картина неоднозначна: с одной стороны, сокращается объём чтения, с другой стороны, возрастает суммарное время декодирования, так как требуется восстановить большое количество значений. В зависимости от того, что является узким местом на конкретной машине (диск или процессор), итоговое время может как уменьшиться, так и остаться близким к исходному.

В сумме по коллекции сжатие уменьшает общий размер координатных блоков, что снижает нагрузку на файловую подсистему и повышает эффективность кэширования. При множественных запросах эффект усиливается, так как большее число востребованных блоков помещается в памяти и кэше, и уменьшается доля времени, потраченная на чтение данных.

## 2.6 Корректность поиска после внедрения сжатия

Поиск после внедрения сжатия остаётся корректным по следующей причине: сжатие меняет только физическое представление данных на диске, но не изменяет математический смысл списков. Корректность обеспечивается выполнением двух

условий: порядок идентификаторов документов в списке терма сохраняется возрастающим; при декодировании полностью восстанавливается исходная последовательность идентификаторов и позиций, то есть применяется обратное преобразование к gar encoding и vbyte. Булев поиск (AND, OR, NOT) опирается на то, что входные списки отсортированы, после чего применяется стандартный алгоритм слияния для пересечения и объединения, а для отрицания строится дополнение относительно множества всех документов. Если восстановленные последовательности совпадают с исходными, результат булевых операций также совпадает с результатом до внедрения сжатия.

## 2.7 Тестирование

Тестирование проводилось в двух направлениях: модульная проверка отдельных компонентов и функциональная проверка всего контура "запрос - выдача".

Модульные тесты проверяют:

- корректность базовых булевых операций над отсортированными списками идентификаторов документов (пересечение, объединение, отрицание);
- корректность токенизации и стабильность обработки параметров токенизации, так как ошибки на этом этапе приводят к расхождениям результатов независимо от сжатия.

Функциональные проверки включают прогоны набора запросов разной сложности:

- запросы из одного терма, чтобы проверить базовое извлечение списка документов;
- конъюнкции и дизъюнкции термов, чтобы проверить корректность операций слияния;
- выражения с отрицанием и скобками, чтобы проверить корректность разбора и вычисления булевого выражения.

### Вывод после прохождения тестов:

```
michael@Huawei:~/InformationSearch/SearchEngine/Lab_6_Compression/tests$ ctest --test-dir build -V
Internal ctest changing into directory: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/build
UpdateCTestConfiguration from
:/home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/build/DartConfiguration.tcl
Parse Config file:/home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/build/DartConfiguration.tcl
UpdateCTestConfiguration from
:/home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/build/DartConfiguration.tcl
```

```

Parse Config file:/home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/build/DartConfiguration.tcl
Test project /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/build
Constructing a list of tests
Done constructing a list of tests
Updating test list for fixtures
Added 0 tests to meet fixture requirements
Checking test dependency graph...
Checking test dependency graph end
test 1
    Start 1: Tokenizer: basic

1: Test command: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/build/unit_tests "Tokenizer:
basic" "-s"
1: Test timeout computed to be: 1500
1: Filters: "Tokenizer: basic"
1: Randomness seeded to: 1437242468
1:
1: ~~~~~
1: unit_tests is a Catch2 v3.5.2 host application.
1: Run with -? for options
1:
1: -----
1: Tokenizer: basic
1: -----
1: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:32
1: .....
1:
1: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:36: PASSED:
1: REQUIRE( out.size() == 3 )
1: with expansion:
1: 3 == 3
1:
1: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:37: PASSED:
1: REQUIRE( out[0] == std::string("python") )
1: with expansion:
1: "python" == "python"
1:
1: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:38: PASSED:
1: REQUIRE( out[1] == std::string("н") )
1: with expansion:
1: "н" == "н"
1:
1: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:39: PASSED:
1: REQUIRE( out[2] == std::string("java") )
1: with expansion:
1: "java" == "java"
1:
1: =====
1: All tests passed (4 assertions in 1 test case)
1:
1/3 Test #1: Tokenizer: basic ..... Passed    0.00 sec
test 2
    Start 2: Tokenizer: keep_numbers

```



```

2: Test command: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/build/unit_tests "Tokenizer:
keep_numbers" "-s"
2: Test timeout computed to be: 1500
2: Filters: "Tokenizer: keep_numbers"
2: Randomness seeded to: 2852549742
2:
2: ~~~~~
2: unit_tests is a Catch2 v3.5.2 host application.
2: Run with -? for options
2:
2: -----
2: Tokenizer: keep_numbers
2: keep_numbers = false
2: -----
2: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:45
2: .....
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:49: PASSED:
2: REQUIRE( out.size() == 2 )
2: with expansion:
2: 2 == 2
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:50: PASSED:
2: REQUIRE( out[0] == std::string("win") )
2: with expansion:
2: "win" == "win"
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:51: PASSED:
2: REQUIRE( out[1] == std::string("abc") )
2: with expansion:
2: "abc" == "abc"
2:
2: -----
2: Tokenizer: keep_numbers
2: keep_numbers = true
2: -----
2: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:54
2: .....
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:58: PASSED:
2: REQUIRE( out.size() == 3 )
2: with expansion:
2: 3 == 3
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:59: PASSED:
2: REQUIRE( out[0] == std::string("win10") )
2: with expansion:
2: "win10" == "win10"
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:60: PASSED:
2: REQUIRE( out[1] == std::string("123") )
2: with expansion:

```

```

2: "123" == "123"
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:61: PASSED:
2: REQUIRE( out[2] == std::string("abc123") )
2: with expansion:
2: "abc123" == "abc123"
2:
2: =====
2: All tests passed (7 assertions in 1 test case)
2:
2/3 Test #2: Tokenizer: keep_numbers ..... Passed 0.00 sec
test 3
  Start 3: Boolean ops

3: Test command: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/build/unit_tests "-s"
3: Test timeout computed to be: 1500
3: Filters: "Boolean ops"
3: Randomness seeded to: 2791290694
3:
3: ~~~~~
3: unit_tests is a Catch2 v3.5.2 host application.
3: Run with -? for options
3:
3: -----
3: Boolean ops
3: AND
3: -----
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:66
3: .....
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:71: PASSED:
3: REQUIRE( r.size == 2 )
3: with expansion:
3: 2 == 2
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:72: PASSED:
3: REQUIRE( r.data[0] == 2u )
3: with expansion:
3: 2 == 2
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:73: PASSED:
3: REQUIRE( r.data[1] == 4u )
3: with expansion:
3: 4 == 4
3:
3: -----
3: Boolean ops
3: OR
3: -----
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:76
3: .....
3:

```

```

3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:81: PASSED:
3: REQUIRE( r.size == 5 )
3: with expansion:
3: 5 == 5
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:82: PASSED:
3: REQUIRE( r.data[0] == 1u )
3: with expansion:
3: 1 == 1
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:83: PASSED:
3: REQUIRE( r.data[1] == 2u )
3: with expansion:
3: 2 == 2
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:84: PASSED:
3: REQUIRE( r.data[2] == 3u )
3: with expansion:
3: 3 == 3
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:85: PASSED:
3: REQUIRE( r.data[3] == 4u )
3: with expansion:
3: 4 == 4
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:86: PASSED:
3: REQUIRE( r.data[4] == 5u )
3: with expansion:
3: 5 == 5
3:
3: -----
3: Boolean ops
3: NOT
3: -----
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:89
3: .....
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:93: PASSED:
3: REQUIRE( r.size == 3 )
3: with expansion:
3: 3 == 3
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:94: PASSED:
3: REQUIRE( r.data[0] == 0u )
3: with expansion:
3: 0 == 0
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:95: PASSED:
3: REQUIRE( r.data[1] == 2u )
3: with expansion:
3: 2 == 2
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_6_Compression/tests/unit_tests.cpp:96: PASSED:

```

```
3: REQUIRE( r.data[2] == 4u )
3: with expansion:
3: 4 == 4
3:
3: =====
3: All tests passed (13 assertions in 1 test case)
3:
3/3 Test #3: Boolean ops ..... Passed 0.00 sec

100% tests passed, 0 tests failed out of 3
```

## 2.8 Проверка построения индексов и сравнение времени

Для подтверждения корректности сборки и воспроизводимости результатов было выполнено построение индекса в двух режимах: базовый вариант без сжатия со сжатием координатных списков.

### Базовый вариант

Построение выполнялось утилитой `index_builder` с параметрами по умолчанию и фиксированным объёмом коллекции. В результате было получено 100000 документов и 1572748 термов, а также рассчитана средняя длина термина в байтах. Итоговые файлы индекса были успешно записаны в каталог `out`, время построения составило 56123 мс.

Пример запуска:

```
./build/index_builder --out out
```

```
docs=100000
```

```
terms=1572748
```

```
avg_term_len_bytes=18.454892
```

```
wrote=out/index.fwd
```

```
wrote=out/index.inv
```

```
time_ms=56123
```

## **Сжатый индекс**

Построение выполнялось утилитой `index_builder6` при тех же входных данных. В результате было получено то же число документов и термов (100000 и 1572748 соответственно), что подтверждает согласованность этапов токенизации и построения словаря. Инвертированный индекс был записан в формате `Lab6` в каталог `out6`; дополнительно утилита выводит размер сериализованных постингов в байтах (`postings_bytes`), так как именно эта часть данных является основной целью сжатия. Время построения составило 53822 мс.

Пример запуска:

```
./build/index_builder6 --out out6
```

```
docs=100000
```

```
terms=1572748
```

```
fwd=out6/index6.fwd
```

```
inv=out6/index6.inv
```

```
postings_bytes=149023061
```

```
time_ms=53822
```

## **Интерпретация результатов**

Совпадение значений `docs` и `terms` в обоих режимах указывает на то, что сжатие не изменяет состав словаря и не влияет на разбиение корпуса на документы и термы, а меняет только способ хранения координатных списков. Полученные времена построения находятся в одном диапазоне, при этом вариант со сжатием показал сопоставимое и немного меньшее время на построение на данном прогоне. Размер

постингов в сжатом варианте фиксируется отдельной метрикой `postings_bytes`, что позволяет далее сравнивать объём данных, которые требуется читать и декодировать во время поиска.

## 2.9 Сравнение результатов поиска

Жирным шрифтом будет выделен текст поисковых запросов

```
michael@Huawei:~/InformationSearch/SearchEngine/Lab_6_Compression/cpp/boolean$  
./build/bool_search --inv out/index.inv --fwd out/index.fwd --q "резерфорд" --top 10
```

hits=48

223 Физика <https://ru.wikipedia.org/wiki/%D0%A4%D0%B8%D0%B7%D0%B8%D0%BA%D0%B0>

309 Атом <https://ru.wikipedia.org/wiki/%D0%90%D1%82%D0%BE%D0%BC>

481 Гелий <https://ru.wikipedia.org/wiki/%D0%93%D0%B5%D0%BB%D0%B8%D0%B9>

1174 1937 год [https://ru.wikipedia.org/wiki/1937\\_%D0%B3%D0%BE%D0%B4](https://ru.wikipedia.org/wiki/1937_%D0%B3%D0%BE%D0%B4)

1596 1908 год [https://ru.wikipedia.org/wiki/1908\\_%D0%B3%D0%BE%D0%B4](https://ru.wikipedia.org/wiki/1908_%D0%B3%D0%BE%D0%B4)

```
michael@Huawei:~/InformationSearch/SearchEngine/Lab_6_Compression/cpp/boolean$ ./build/bool_search6 --inv  
out6/index6.inv --fwd out6/index6.fwd --q "резерфорд" --top 10
```

hits=48

223 Физика <https://ru.wikipedia.org/wiki/%D0%A4%D0%B8%D0%B7%D0%B8%D0%BA%D0%B0>

309 Атом <https://ru.wikipedia.org/wiki/%D0%90%D1%82%D0%BE%D0%BC>

481 Гелий <https://ru.wikipedia.org/wiki/%D0%93%D0%B5%D0%BB%D0%B8%D0%B9>

1174 1937 год [https://ru.wikipedia.org/wiki/1937\\_%D0%B3%D0%BE%D0%B4](https://ru.wikipedia.org/wiki/1937_%D0%B3%D0%BE%D0%B4)

1596 1908 год [https://ru.wikipedia.org/wiki/1908\\_%D0%B3%D0%BE%D0%B4](https://ru.wikipedia.org/wiki/1908_%D0%B3%D0%BE%D0%B4)

Для проверки того, что внедрение сжатия не изменило семантику поиска, было выполнено сравнение результатов выдачи для одинакового запроса на двух вариантах индекса: базовом (без сжатия) и варианте лабораторной работы 6 (со сжатием).

В качестве тестового запроса был выбран терм резерфорд. Поиск выполнялся утилитами **bool\_search** и **bool\_search6**. В обоих случаях поисковая система вернула одинаковое количество найденных документов: hits=48. Также совпали первые

результаты выдачи, включая идентификаторы документов, заголовки и ссылки (например, документы "Физика", "Атом", "Гелий" и т.д.)

Полученное совпадение подтверждает, что сжатие влияет только на способ хранения координатных списков на диске и не изменяет содержимое списков, которые используются при вычислении результата запроса. Следовательно, при корректной реализации декодирования и при сохранении упорядоченности списков булевы операции применяются к тем же данным, что и в базовом варианте, и формируют идентичную поисковую выдачу. Для наглядности ниже приведем еще пару примеров.

```
michael@Huawei:~/InformationSearch/SearchEngine/Lab_6_Compression/cpp/boolean$ ./build/bool_search --inv out/index.inv --fwd out/index.fwd --q "ssh" --top 10
```

```
hits=143
```

```
154 Интернет
```

```
https://ru.wikipedia.org/wiki/%D0%98%D0%BD%D1%82%D0%B5%D1%80%D0%BD%D0%B5%D1%82
```

```
532 FTP https://ru.wikipedia.org/wiki/FTP
```

```
553 SSH https://ru.wikipedia.org/wiki/SSH
```

```
554 TCP/IP https://ru.wikipedia.org/wiki/TCP/IP
```

```
556 Telnet https://ru.wikipedia.org/wiki/Telnet
```

```
michael@Huawei:~/InformationSearch/SearchEngine/Lab_6_Compression/cpp/boolean$ ./build/bool_search6 --inv out6/index6.inv --fwd out6/index6.fwd --q "ssh" --top 10
```

```
hits=143
```

```
154 Интернет
```

```
https://ru.wikipedia.org/wiki/%D0%98%D0%BD%D1%82%D0%B5%D1%80%D0%BD%D0%B5%D1%82
```

```
532 FTP https://ru.wikipedia.org/wiki/FTP
```

```
553 SSH https://ru.wikipedia.org/wiki/SSH
```

```
554 TCP/IP https://ru.wikipedia.org/wiki/TCP/IP
```

```
556 Telnet https://ru.wikipedia.org/wiki/Telnet
```

```
michael@Huawei:~/InformationSearch/SearchEngine/Lab_6_Compression/cpp/boolean$ ./build/bool_search --inv out/index.inv --fwd out/index.fwd --q "материнская && плата" --top 10
```

```
hits=24
```

```
146 Персональный компьютер
```

```
https://ru.wikipedia.org/wiki/%D0%9F%D0%B5%D1%80%D1%81%D0%BE%D0%BD%D0%B0%D0%BB%D1%8C%D0%BD%D1%8B%D0%B9_%D0%BA%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80
```

270 Компьютерный сленг

[https://ru.wikipedia.org/wiki/%D0%9A%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80%D0%BD%D1%8B%D0%B9\\_%D1%81%D0%BB%D0%B5%D0%BD%D0%B3](https://ru.wikipedia.org/wiki/%D0%9A%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80%D0%BD%D1%8B%D0%B9_%D1%81%D0%BB%D0%B5%D0%BD%D0%B3)

5626 IBM PC [https://ru.wikipedia.org/wiki/IBM\\_PC](https://ru.wikipedia.org/wiki/IBM_PC)

8422 Центральный процессор

[https://ru.wikipedia.org/wiki/%D0%A6%D0%B5%D0%BD%D1%82%D1%80%D0%B0%D0%BB%D1%8C%D0%BD%D1%8B%D0%B9\\_%D0%BF%D1%80%D0%BE%D1%86%D0%B5%D1%81%D1%81%D0%BE%D1%80](https://ru.wikipedia.org/wiki/%D0%A6%D0%B5%D0%BD%D1%82%D1%80%D0%B0%D0%BB%D1%8C%D0%BD%D1%8B%D0%B9_%D0%BF%D1%80%D0%BE%D1%86%D0%B5%D1%81%D1%81%D0%BE%D1%80)

9339 Xeon <https://ru.wikipedia.org/wiki/Xeon>

**michael@Huawei:~/InformationSearch/SearchEngine/Lab\_6\_Compression/cpp/boolean\$ ./build/bool\_search6 --inv out6/index6.inv --fwd out6/index6.fwd --q "материнская && плата" --top 10**  
hits=24

146 Персональный компьютер

[https://ru.wikipedia.org/wiki/%D0%9F%D0%B5%D1%80%D1%81%D0%BE%D0%BD%D0%B0%D0%BB%D1%8C%D0%BD%D1%8B%D0%B9\\_%D0%BA%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80](https://ru.wikipedia.org/wiki/%D0%9F%D0%B5%D1%80%D1%81%D0%BE%D0%BD%D0%B0%D0%BB%D1%8C%D0%BD%D1%8B%D0%B9_%D0%BA%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80)

270 Компьютерный сленг

[https://ru.wikipedia.org/wiki/%D0%9A%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80%D0%BD%D1%8B%D0%B9\\_%D1%81%D0%BB%D0%B5%D0%BD%D0%B3](https://ru.wikipedia.org/wiki/%D0%9A%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80%D0%BD%D1%8B%D0%B9_%D1%81%D0%BB%D0%B5%D0%BD%D0%B3)

5626 IBM PC [https://ru.wikipedia.org/wiki/IBM\\_PC](https://ru.wikipedia.org/wiki/IBM_PC)

8422 Центральный процессор

[https://ru.wikipedia.org/wiki/%D0%A6%D0%B5%D0%BD%D1%82%D1%80%D0%B0%D0%BB%D1%8C%D0%BD%D1%8B%D0%B9\\_%D0%BF%D1%80%D0%BE%D1%86%D0%B5%D1%81%D1%81%D0%BE%D1%80](https://ru.wikipedia.org/wiki/%D0%A6%D0%B5%D0%BD%D1%82%D1%80%D0%B0%D0%BB%D1%8C%D0%BD%D1%8B%D0%B9_%D0%BF%D1%80%D0%BE%D1%86%D0%B5%D1%81%D1%81%D0%BE%D1%80)

9339Xeon <https://ru.wikipedia.org/wiki/Xeon>



### 3. Выводы

В ходе лабораторной работы было реализовано сжатое хранение инвертированного индекса на основе кодирования разностей и переменного-байтового кодирования целых чисел. Это позволило уменьшить объём координатных списков, сохранив их логическое содержимое и порядок следования идентификаторов документов и позиций вхождений. Влияние сжатия зависит от частотности термов. Для редких термов накладные расходы на декодирование минимальны, так как списки короткие, при этом экономия места достигается сразу. Для термов средней частотности обычно получается наиболее выгодный компромисс: объём чтения с диска уменьшается, а затраты на декодирование остаются умеренными.

Для высокочастотных термов выигрыш по размеру максимален, однако возрастает суммарное время декодирования из-за большого числа восстанавливаемых значений, поэтому итоговая скорость определяется балансом между вводом-выводом и вычислениями. Корректность поиска после внедрения сжатия обоснована тем, что сжатие изменяет только физическое представление данных, а при декодировании полностью восстанавливаются исходные последовательности.

Булевы операции AND, OR, NOT применяются к отсортированным спискам документов стандартными алгоритмами слияния, поэтому при корректном восстановлении списков результат поиска совпадает с результатом для несжатого представления. В рамках выполнения работы были закреплены навыки проектирования бинарного формата хранения, организации последовательной сериализации данных и построения обратного декодирования. Также были получены практические навыки тестирования поискового контура: проверки корректности базовых операций над списками результатов и функциональной проверки обработки запросов различной структуры.

## Список литературы

- [1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д.А. Ключина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))
- [2] Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. 3-е изд. М.: ООО "И.Д. Вильямс", 2013.
- [3] Кнут Д. Э. Искусство программирования. Том 3. Сортировка и поиск. М.: ООО "И.Д. Вильямс", 2007.
- [4] Тененбаум Э., Лангсам И., Одженстейн М. Структуры данных с помощью С. М.: Вильямс, 2000.
- [5] Ватолин Д., Ратушняк А., Смирнов М., Юкин В. Методы сжатия данных. Устройство архиваторов, сжатие изображений и видео.